Solución del Taller 1

Teoría de Aprendizaje de Máquina – 2026-2
Clasificación supervisada, vecinos cercanos y evaluación de modelos

Desarrollo matemático y computacional

9 de noviembre de 2026

Alcance de la solución

Se desarrollan los incisos 1.1–1.3, 2.1–2.3 y 3.1–3.3. Los procedimientos manuales se muestran paso a paso. Los experimentos que requieren programación se implementan en el cuaderno `Taller_TAM_Colab.ipynb`, compatible con Google Colab. Los resultados numéricos de `make_moons` corresponden a los parámetros y semillas indicados en el enunciado.

Índice

1. Punto 1. Clasificación lineal y algoritmo del perceptrón **2**

1.1. Inciso 1.1. Frontera de decisión . . . 2
1.2. Inciso 1.2. Actualización del perceptrón . . . 3
1.3. Inciso 1.3. Ejercicio numérico . . . 4

2. Punto 2. Clasificación por k vecinos más cercanos (k-NN) **4**

2.1. Inciso 2.1. Cálculo manual . . . 5
2.2. Inciso 2.2. Implementación con `make_moons` . . . 5
2.3. Inciso 2.3. Interpretación . . . 7

3. Punto 3. Evaluación de un clasificador binario **7**

3.1. Inciso 3.1. Matriz de confusión . . . 7
3.2. Inciso 3.2. Cálculo e interpretación de métricas . . . 8
3.3. Inciso 3.3. Verificación con Python y limitaciones del accuracy . . . 9

4. Conclusiones **9**

1. Punto 1. Clasificación lineal y algoritmo del perceptrón

Se considera un clasificador binario con observaciones $\mathbf{x} \in \mathbb{R}^2$ y etiquetas $y \in \{-1, +1\}$. La función de decisión es

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b, \qquad \widehat{y} = \begin{cases} +1, & f(\mathbf{x}) > 0, \\ -1, & f(\mathbf{x}) \leq 0. \end{cases}$$

El vector $\mathbf{w}$ determina la orientación de la frontera y b desplaza dicha frontera respecto al origen.

1.1. Inciso 1.1. Frontera de decisión

Se proporcionan

$$\mathbf{w} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad b = -3.$$

La frontera de decisión se obtiene igualando a cero la función de decisión:

$$\begin{aligned} \mathbf{w}^T\mathbf{x} + b &= 0, \\ 1x_1 + 2x_2 - 3 &= 0. \end{aligned}$$

Por tanto, la ecuación de la recta es

$$\boxed{x_1 + 2x_2 - 3 = 0} \qquad \Longleftrightarrow \qquad \boxed{x_2 = \frac{3 - x_1}{2}}.$$

Para clasificar cada observación se evalúa $f(\mathbf{x}) = x_1 + 2x_2 - 3$:

Observación	Evaluación	Signo de $f(\mathbf{x})$	Predicción
$\mathbf{x}_1 = (0, 0)$	$0 + 2(0) - 3 = -3$	Negativo	$\widehat{y}_1 = -1$
$\mathbf{x}_2 = (1, 0)$	$1 + 2(0) - 3 = -2$	Negativo	$\widehat{y}_2 = -1$
$\mathbf{x}_3 = (2, 1)$	$2 + 2(1) - 3 = 1$	Positivo	$\widehat{y}_3 = +1$
$\mathbf{x}_4 = (3, 1)$	$3 + 2(1) - 3 = 2$	Positivo	$\widehat{y}_4 = +1$

Las observaciones con $f(\mathbf{x}) > 0$ quedan en la región de clase $+1$. Las que cumplen $f(\mathbf{x}) \leq 0$ quedan en la región de clase -1. En particular, un punto situado exactamente sobre la recta se asignaría a la clase -1, de acuerdo con la regla del enunciado.

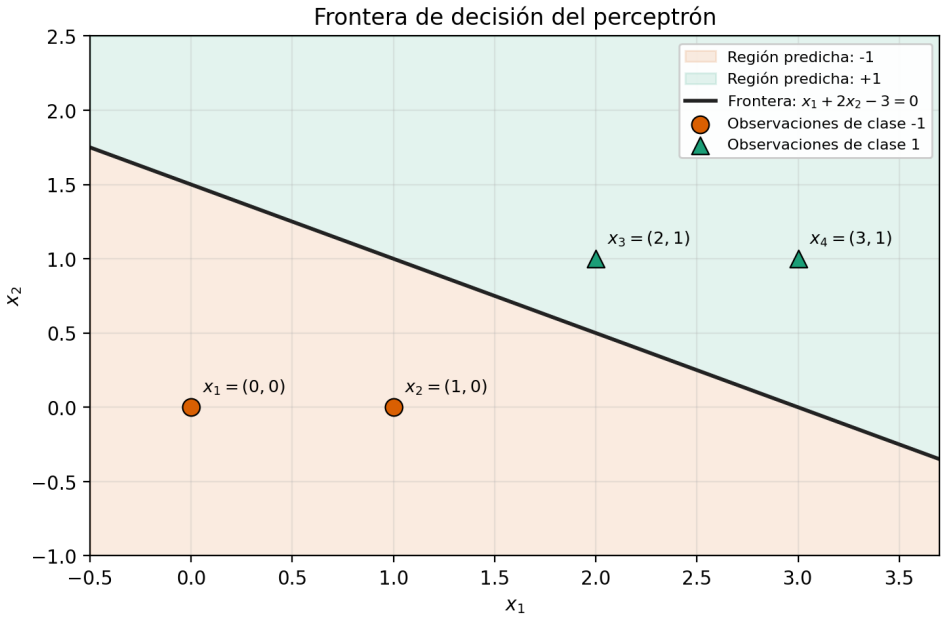


Figura 1: Frontera de decisión y clasificación de las cuatro observaciones del inciso 1.1.

1.2. Inciso 1.2. Actualización del perceptrón

La regla de actualización para una observación mal clasificada $(\mathbf{x}_i, y_i)$ es

$$\mathbf{w}_{\text{nuevo}} = \mathbf{w} + \eta y_i \mathbf{x}_i, \qquad b_{\text{nuevo}} = b + \eta y_i, \qquad \eta > 0.$$

El margen de esa observación se define como

$$m_i = y_i(\mathbf{w}^T \mathbf{x}_i + b).$$

Se debe demostrar que, después de actualizar los parámetros usando esa misma observación,

$$m_{i,\text{nuevo}} = m_i + \eta(\|\mathbf{x}_i\|_2^2 + 1).$$

Demostración. Por definición del margen con los nuevos parámetros,

$$m_{i,\text{nuevo}} = y_i \left(\mathbf{w}_{\text{nuevo}}^T \mathbf{x}_i + b_{\text{nuevo}} \right).$$

Sustituyendo las reglas de actualización,

$$\begin{aligned} m_{i,\text{nuevo}} &= y_i \left[(\mathbf{w} + \eta y_i \mathbf{x}_i)^T \mathbf{x}_i + b + \eta y_i \right] \\ &= y_i \left[\mathbf{w}^T \mathbf{x}_i + b + \eta y_i \mathbf{x}_i^T \mathbf{x}_i + \eta y_i \right] \\ &= y_i(\mathbf{w}^T \mathbf{x}_i + b) + \eta y_i^2 \mathbf{x}_i^T \mathbf{x}_i + \eta y_i^2. \end{aligned}$$

Como $y_i \in \{-1, +1\}$, se tiene $y_i^2 = 1$. Además, $\mathbf{x}_i^T \mathbf{x}_i = \|\mathbf{x}_i\|_2^2$. Por consiguiente,

$$\begin{aligned} m_{i,\text{nuevo}} &= m_i + \eta \|\mathbf{x}_i\|_2^2 + \eta \\ &= \boxed{m_i + \eta(\|\mathbf{x}_i\|_2^2 + 1)}. \end{aligned}$$

La igualdad queda demostrada.

Interpretación. Puesto que $\eta > 0$ y $\|\mathbf{x}_i\|_2^2 + 1 > 0$, el margen de la observación utilizada en la actualización aumenta estrictamente:

$$m_{i,\text{nuevo}} - m_i = \eta(\|\mathbf{x}_i\|_2^2 + 1) > 0.$$

Esto significa que la actualización mueve la función de decisión en una dirección favorable a la etiqueta correcta de esa observación. No implica, sin embargo, que todas las demás observaciones queden correctamente clasificadas ni que el margen corregido sea necesariamente positivo después de una sola actualización.

1.3. Inciso 1.3. Ejercicio numérico

Se tiene

$$\mathbf{w} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad b = 0, \quad \eta = 0.5, \quad \mathbf{x} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad y = +1.$$

Primero se calcula la función de decisión:

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b = 0.$$

Como la regla asigna la clase -1 cuando $f(\mathbf{x}) \leq 0$, la predicción es $\widehat{y} = -1$. La etiqueta real es $y = +1$, así que la observación está mal clasificada y se debe actualizar.

Para los pesos:

$$\begin{aligned} \mathbf{w}_{\text{nuevo}} &= \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 0.5(1) \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\ &= \boxed{\begin{bmatrix} 1 \\ 0.5 \end{bmatrix}}. \end{aligned}$$

Para el sesgo:

$$b_{\text{nuevo}} = 0 + 0.5(1) = \boxed{0.5}.$$

El margen inicial es $m = yf(\mathbf{x}) = 0$. El nuevo margen se calcula como

$$\begin{aligned} m_{\text{nuevo}} &= y(\mathbf{w}_{\text{nuevo}}^T\mathbf{x} + b_{\text{nuevo}}) \\ &= 1\,(1 \cdot 2 + 0.5 \cdot 1 + 0.5) \\ &= \boxed{3}. \end{aligned}$$

También se verifica directamente con la expresión demostrada:

$$m_{\text{nuevo}} = 0 + 0.5\left((2^2 + 1^2) + 1\right) = 3.$$

Por tanto, sí corresponde actualizar y los parámetros nuevos son $\mathbf{w}_{\text{nuevo}} = [1, 0.5]^T$, $b_{\text{nuevo}} = 0.5$ y $m_{\text{nuevo}} = 3$.

2. Punto 2. Clasificación por k vecinos más cercanos (k-NN)

El algoritmo k-NN clasifica una observación nueva a partir de las etiquetas de sus k vecinos de entrenamiento más cercanos. En este taller se utiliza la distancia euclidiana:

$$d(\mathbf{q}, \mathbf{x}_i) = \sqrt{(q_1 - x_{i,1})^2 + (q_2 - x_{i,2})^2}.$$

2.1. Inciso 2.1. Cálculo manual

La observación por clasificar es $\mathbf{q} = (1, 1)$. Leyendo la tabla del enunciado, el conjunto de entrenamiento es:

Observación	x_1	x_2	x_3	x_4	x_5
Coordenadas	$(0, 0)$	$(1, 0)$	$(3, 1)$	$(0, 3)$	$(1.1, 1.1)$
Clase	A	A	B	B	B

En particular, se interpreta $\mathbf{x}_5 = (1.1, 1.1)$, tal como aparece en la tabla del PDF original.

Las distancias son:

$$
\begin{aligned}
d(\mathbf{q}, \mathbf{x}_1) &= \sqrt{(1-0)^2 + (1-0)^2} = \sqrt{2} \approx 1.4142, \\
d(\mathbf{q}, \mathbf{x}_2) &= \sqrt{(1-1)^2 + (1-0)^2} = 1, \\
d(\mathbf{q}, \mathbf{x}_3) &= \sqrt{(1-3)^2 + (1-1)^2} = 2, \\
d(\mathbf{q}, \mathbf{x}_4) &= \sqrt{(1-0)^2 + (1-3)^2} = \sqrt{5} \approx 2.2361, \\
d(\mathbf{q}, \mathbf{x}_5) &= \sqrt{(1-1.1)^2 + (1-1.1)^2} = \sqrt{0.02} \approx 0.1414.
\end{aligned}
$$

Ordenando de menor a mayor:

Orden	Observación	Distancia	Clase
1	x_5	0.1414	B
2	x_2	1.0000	A
3	x_1	1.4142	A
4	x_3	2.0000	B
5	x_4	2.2361	B

Clasificación con $k = 1$. El vecino más cercano es x_5, cuya clase es B. Por tanto, $\boxed{\widehat{c}(\mathbf{q}) = B}$.

Clasificación con $k = 3$. Los tres vecinos son x_5 (B), x_2 (A) y x_1 (A). Hay dos votos para A y uno para B, por lo que $\boxed{\widehat{c}(\mathbf{q}) = A}$.

Clasificación con $k = 5$. Se consideran las cinco observaciones: A tiene dos votos y B tiene tres. Por tanto, $\boxed{\widehat{c}(\mathbf{q}) = B}$.

El valor de k puede cambiar la predicción porque modifica cuántas observaciones participan en la votación. Con un k pequeño predominan los vecinos muy locales; al aumentar k, también influyen observaciones más lejanas, que pueden pertenecer a otra clase.

2.2. Inciso 2.2. Implementación con `make_moons`

Se generan $N = 300$ observaciones bidimensionales con `noise` $= 0.20$ y `random_state` $= 42$. Después se utiliza una partición estratificada de 70 % para entrenamiento y 30 % para prueba. La estratificación conserva la proporción de clases en ambos subconjuntos.

El siguiente código resume el experimento; el cuaderno de Colab adjunto contiene el flujo completo, las figuras y las verificaciones:

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

```
X, y = make_moons(n_samples=300, noise=0.20, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

for k in [1, 5, 15]:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    print(k, model.score(X_train, y_train),
             model.score(X_test, y_test))
```

La partición obtenida contiene 210 observaciones de entrenamiento y 90 de prueba. Como las dos clases se generan con igual cantidad de ejemplos, la partición contiene 105 observaciones de cada clase en entrenamiento y 45 de cada clase en prueba.

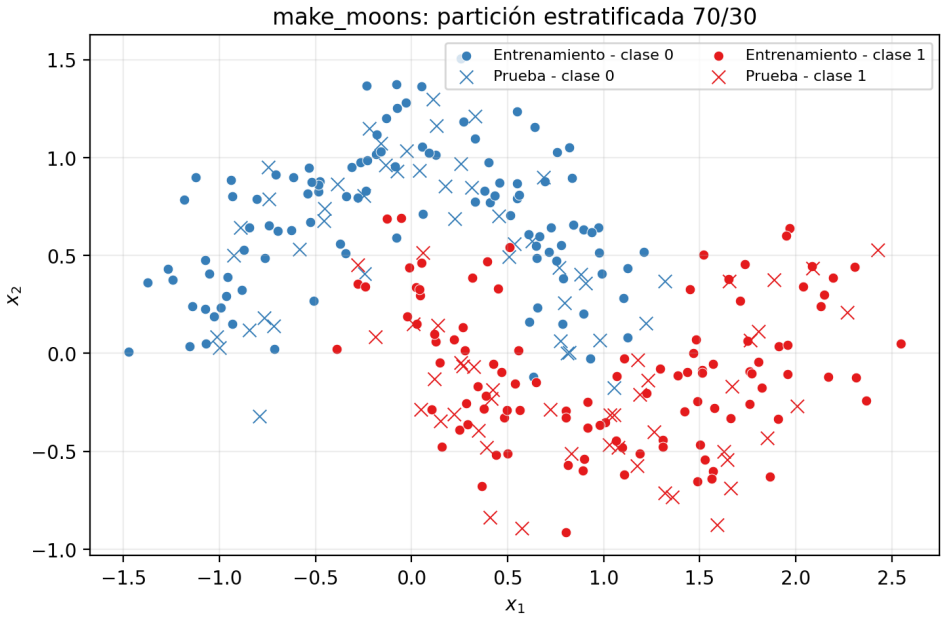


Figura 2: Datos generados con `make_moons` y división estratificada entre entrenamiento y prueba.

Los accuracy obtenidos son:

k	Accuracy de entrenamiento	Accuracy de prueba
1	1.0000 (100.00 %)	0.9333 (93.33 %)
5	0.9571 (95.71 %)	0.9556 (95.56 %)
15	0.9571 (95.71 %)	0.9333 (93.33 %)

Las regiones de decisión para $k = 1$ y $k = 15$ se muestran en la Figura 3. En ellas, cada región del plano representa la clase que el modelo asignaría a una observación ubicada en esa zona.

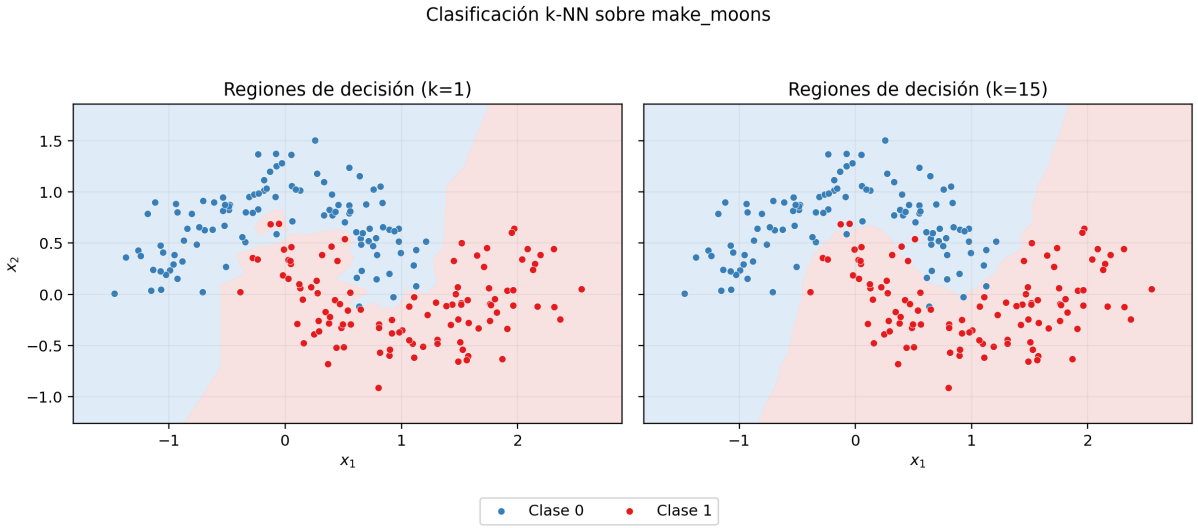


Figura 3: Regiones de decisión de k-NN para $k = 1$ y $k = 15$, entrenadas con el mismo subconjunto de entrenamiento.

2.3. Inciso 2.3. Interpretación

- **Valor pequeño de k.** La decisión depende de muy pocos vecinos. Para $k = 1$, cada observación de entrenamiento es su propio vecino más cercano, por lo que el accuracy de entrenamiento puede llegar a 100 %. La frontera puede ser irregular y sensible al ruido, lo que aumenta el riesgo de sobreajuste.
- **Valor grande de k.** La votación incorpora una región más amplia. La frontera tiende a ser más suave, pero puede ignorar estructuras locales y favorecer el subajuste si k es excesivo.

En esta evaluación concreta, $k = 5$ presenta el mayor accuracy de prueba (95.56 %) y una diferencia pequeña entre entrenamiento (95.71 %) y prueba (95.56 %). Por ello, entre los tres valores solicitados, es el que mostró el mejor desempeño de generalización en esta partición.

¿Por qué no seleccionar formalmente k con el conjunto de prueba? El conjunto de prueba debe reservarse para obtener una estimación final del rendimiento sobre datos no utilizados durante el desarrollo. Si se prueban muchos valores de k y se escoge repetidamente el que maximiza el accuracy de prueba, la selección termina adaptándose indirectamente a ese conjunto. Una selección formal se realiza mediante validación, por ejemplo, validación cruzada sobre los datos de entrenamiento. Después se ajusta el modelo elegido con los datos de entrenamiento disponibles y se evalúa una sola vez sobre el conjunto de prueba reservado.

3. Punto 3. Evaluación de un clasificador binario

Se tienen las etiquetas verdaderas y las predicciones:

$$\mathbf{y} = [0, 0, 0, 0, 1, 1, 1, 1, 1, 1], \qquad \widehat{\mathbf{y}} = [0, 1, 0, 0, 1, 0, 1, 1, 0, 1].$$

La clase positiva es 1. Las filas de la matriz de confusión representan las clases reales y las columnas las clases predichas.

3.1. Inciso 3.1. Matriz de confusión

Se comparan las diez parejas de valores:

Etiqueta real	Predicción	Conteo	Tipo
0	0	3	Verdadero negativo (TN)
0	1	1	Falso positivo (FP)
1	0	2	Falso negativo (FN)
1	1	4	Verdadero positivo (TP)

Por tanto,

$$TN = 3, \qquad FP = 1, \qquad FN = 2, \qquad TP = 4.$$

La matriz de confusión queda

	$\widehat{y} = 0$	$\widehat{y} = 1$
$y = 0$	3	1
$y = 1$	2	4

3.2. Inciso 3.2. Cálculo e interpretación de métricas

Exactitud (Accuracy). Mide la proporción de predicciones correctas sobre el total:

$$\begin{aligned}\text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ &= \frac{4+3}{4+3+1+2} = \boxed{0.70 = 70\,\%}.\end{aligned}$$

Precisión (Precision). De todas las observaciones predichas como positivas, mide qué proporción realmente es positiva:

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{4}{4+1} = \boxed{0.80 = 80\,\%}.$$

Exhaustividad o sensibilidad (Recall). De todas las observaciones positivas reales, mide qué proporción fue detectada:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{4}{4+2} = \boxed{\frac{2}{3} \approx 0.6667 = 66.67\,\%}.$$

Medida F1. Es la media armónica entre precision y recall; resume el equilibrio entre ambas:

$$\begin{aligned}F_1 &= 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \\ &= 2\frac{(0.8)(2/3)}{0.8 + 2/3} = \boxed{\frac{8}{11} \approx 0.7273 = 72.73\,\%}.\end{aligned}$$

Métrica	Resultado
Accuracy	70.00 %
Precision	80.00 %
Recall	66.67 %
F_1	72.73 %

En este caso, el modelo acierta el 70 % del total. De las predicciones positivas, el 80 % es correcto; sin embargo, el modelo detecta solamente dos tercios de los positivos reales. La precision es mayor que el recall, lo que indica que el modelo comete relativamente pocos falsos positivos, pero deja sin detectar dos de los seis positivos reales.

3.3. Inciso 3.3. Verificación con Python y limitaciones del accuracy

La comprobación con scikit-learn se realiza con:

```
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score
)

y = [0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
y_pred = [0, 1, 0, 0, 1, 0, 1, 1, 0, 1]

print(confusion_matrix(y, y_pred))
print("Accuracy:", accuracy_score(y, y_pred))
print("Precision:", precision_score(y, y_pred))
print("Recall:", recall_score(y, y_pred))
print("F1:", f1_score(y, y_pred))
```

La salida esperada es:

```
[[3 1]
 [2 4]]
Accuracy: 0.7
Precision: 0.8
Recall: 0.6666666666666666
F1: 0.7272727272727272
```

Dos situaciones en las que accuracy por sí sola puede ser engañosa son:

1. **Clases desbalanceadas.** Considérese un conjunto con 990 observaciones normales y 10 fraudulentas. Un clasificador que prediga siempre la clase normal obtiene 99 % de accuracy, pero recall igual a cero para la clase fraude. La exactitud alta no refleja que el objetivo principal –detectar fraudes– no se cumple.
2. **Costos diferentes de los errores.** En un sistema de apoyo al diagnóstico, un falso negativo (no detectar una enfermedad presente) puede tener consecuencias más graves que un falso positivo (solicitar una prueba adicional a una persona sana). Accuracy cuenta ambos errores sin distinguir sus costos. Por eso deben analizarse métricas como recall, precision y la matriz de confusión, de acuerdo con el problema.

4. Conclusiones

1. El perceptrón utiliza una función lineal $\mathbf{w}^T\mathbf{x} + b$ para separar dos clases. La actualización incrementa estrictamente el margen de la observación corregida, aunque no garantiza por sí sola la clasificación correcta de todo el conjunto.
2. k-NN clasifica mediante proximidad y votación. El parámetro k controla el nivel de localidad de la decisión: valores pequeños son sensibles a observaciones individuales, mientras que valores grandes suavizan la frontera. En el experimento solicitado, $k = 5$ obtuvo el mayor accuracy de prueba (95.56 %).
3. La evaluación de un clasificador requiere más de una métrica. Accuracy mide aciertos globales, precision evalúa la confiabilidad de las predicciones positivas, recall mide la detección de positivos reales y F1 resume el equilibrio entre precision y recall.
4. La semilla aleatoria y la separación explícita entre entrenamiento y prueba hacen reproducible la comparación. La selección de hiperparámetros debe realizarse con validación, reservando el conjunto de prueba para la evaluación final.